IPA Project Report (Phase I)
Implementation of a 64-bit RISC-V CPU

Ritvik Avula
Roll No: 2024112019

Pavani Malchuru
Roll No: 2024102043

Nikhilesh Nallavelli
Roll No: 2024102051

***Abstract*—This report presents the design and implementation of a 64-bit RISC-V CPU using Verilog. We provide a comprehensive overview of the architecture used - from elaborating the design of each of the modules that constitute the CPU, to how the datapath was designed. For Phase 1, we implemented a CPU that processes instructions in a sequential order.**

I. Introduction and Background

THE RISC-V ISA is an open source instruction set architecture that focuses on maintaining simple hardware and using basic instructions in order to design and program more complicated entities. This simplicity makes RISC-V an ideal choice for custom processor design. The instruction set is modular, allowing implementations to support only the necessary base instructions (RV64I) or extend with optional extensions for floating-point operations, atomic instructions, and more. In this project, we focus on implementing the RV64I base integer instruction set for 64-bit systems.

Of all the instructions in the base RV64I ISA, we've implemented the following subset of instructions:

- **R-type** (add, add/or/xor, sll/srl/sra)
- **I-type** (addi and etc..., ld, jalr)
- **S-type** (sd)
- **B-type** (beq)
- **J-type** (jal)

II. Architecture and Design

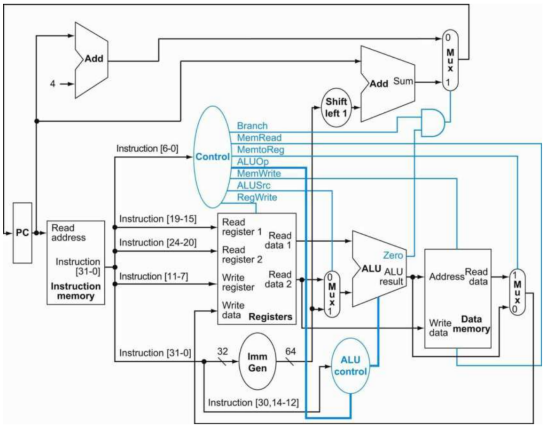

Fig. 1: High-level architecture of the RV64I sequential processor. Courtesy of *Computer Organization and Design, Patterson and Hennessy* [1]

The RV64I CPU consists of several key modules that work in conjunction to execute instructions. Fig. 1 illustrates the high-level architecture of the processor.

The datapath consists of the following main components:

1) **Program Counter (PC):** Stores the address of the current instruction and increments to fetch the next instruction
2) **Instruction Memory:** Stores the program instructions
3) **Register File:** Contains 32 general-purpose 64-bit registers
4) **Arithmetic Logic Unit (ALU):** Performs computations - either arithmetic or to declare flags to be used in conditional operations
5) **Control Unit:** Decodes and instruction and generates control signals that determine which modules should be active during the instruction cycle
6) **Immediate Generator:** Extracts and extends immediate values from instructions

The following subsections go into more detail about each of the modules.

A. Program Counter (PC)

Inputs: `clk, reset, [63:0] pc_in`
Outputs: `[63:0] pc_out`

The PC module manages instruction sequencing. It maintains the current program counter value and implements control logic to either increment to the next instruction or jump to a target address. The module is designed as a 64-bit register with a synchronous update mechanism triggered by the clock.

B. Instruction Memory

Inputs: `clk, reset, [63:0] pc_curr`
Outputs: `[31:0] inst`

The Instruction Memory module stores the program that we want to execute. These instructions are 32 bits wide, and the current instruction is determined by the value of the PC, which as discussed earlier, is 64 bits wide.

To function, the module first initializes its memory using instructions from the file "'instructions.txt"', which has a byte of instruction per line - meaning four lines in conjunction make up one specific instruction completely. The module then reads instructions from instruction memory based on the current PC value and outputs the fetched 32-bit instruction to the Control Block, Register File, and the Immediate Generator.

C. Register File

Inputs: `clk, reset, reg_write_en, [4:0] read_reg1, read_reg2, write_reg, [63:0] write_data`

Outputs: `[63:0] read_data1, read_data2`

The register file contains 32 general-purpose 64-bit registers (x0 to x31), where x0 is hardwired to zero. In general, the RISC-V registers are generally assigned for the operations shown in Fig. 2.

Register File

Register	Name	Convention	Saver
x0	zero	constant 0	na
x1	ra	return address	caller
x2	sp	stack pointer	callee
x3	gp	global pointer	na
x4	tp	thread pointer	na
x5-x7	t0-t2	temporary	caller
x8	s0/fp	saved/frame ptr	callee
x9	s1	saved	callee
x10-x11	a0-a1	arg/return val	caller
x12-x17	a2-a7	arguments	caller
x18-x27	s2-s11	saved	callee
x28-x31	t3-t6	temporary	caller

Fig. 2: The RISC-V convention for how registers x0 to x31 are usually used.

The register file that we implemented has the following features.

- Asynchronous dual-read ports for fetching operands
- One write port for storing results back in registers
- Control signals to enable/disable writing

To improve power efficiency, we implemented a 'lazy read' condition for x0 - allowing writing to x0 but clearing it to 0 only when it's being read.

D. Arithmetic Logic Unit (ALU)

Inputs: `[63:0] rs1, rs2 [3:0] alu_ctrl`

Outputs: `[63:0] result, cout, carry_flag, overflow_flag, zero_flag`

The ALU is a complex module responsible for executing arithmetic and logical operations. The flags it generates are used for evaluating conditional statements and determine validity of results. A brief description of the flags is as follows:

- **Carry flag:** Indicates an unsigned overflow occurred during addition or subtraction. Set when the result exceeds the 64-bit range in unsigned arithmetic. Irrelevant for signed arithmetic.
- **Overflow flag:** Indicates a signed overflow occurred. Set when the result of a signed arithmetic operation exceeds the valid 64-bit signed range (-2^{63} to $2^{63}-1$). Irrelevant for unsigned arithmetic.
- **Zero flag:** Set to 1 when the ALU result is zero. This is mainly used for conditional branch instructions (i.e, instructions like `beq`) to determine equality conditions.

An ALU is further constituted of a few major components of its own.

1) 64-bit Adder/Subtractor: We used a ripple carry adder which can do both add and subtract operations. The general structure is shown in Fig. 3.

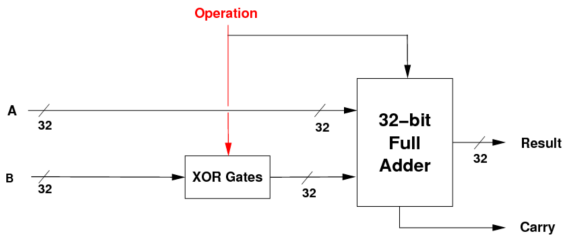

Fig. 3: The adder block structure. The figure shows operation on 32-bit numbers, while we have extended it to 64 bits.

The cin input that is usually given to full adders is used here as an operation selector; to replicate 2s complement subtraction, we do a + (!b) + 1.

2) 64-bit Logic Gates: Implements bitwise logical operations like AND, OR, XOR, and NOR. Each operation is performed bitwise across all 64 bits.

3) 64-bit Barrel Shifter: Supports logical left shifts, logical right shifts, and arithmetic right shifts. The shift amount is determined only by the least significant 6 bits of rs2 (allowing shifts up to 63 positions). Shifts above that are wrapped back - in the sense that a bit shift of 64 bits would be the same as shifting by 1 bit.

Fig. 3 shows the general structure of a barrel shifter.

To generalize this for both right and left shifts, we implemented a row of muxes before and after the circuit shown in Fig. 4. Based on the opcode, the inputs would either be given in little endian or big endian format, and collected at output likewise. This is because shifting right is technically the same as shifting the bit-reversed input left.

Right shifts are also subdivided between logical and arithmetic shifts. To fix this, instead of hardwiring the bottom muxes to zeros as shown in the figure, we instead used the opcode to determine whether to give 0 (for logical shift) or 1 (for arithmetic shift)

E. Immediate Generator (Imm_gen)

This module extracts immediate values from the instruction and performs sign extension to 64 bits. It handles multiple RISC-V instruction formats:

- **I-type:** 12-bit signed immediates (instructions: ADDI, LW, etc.)
- **S-type:** 12-bit signed immediates (instructions: SW)

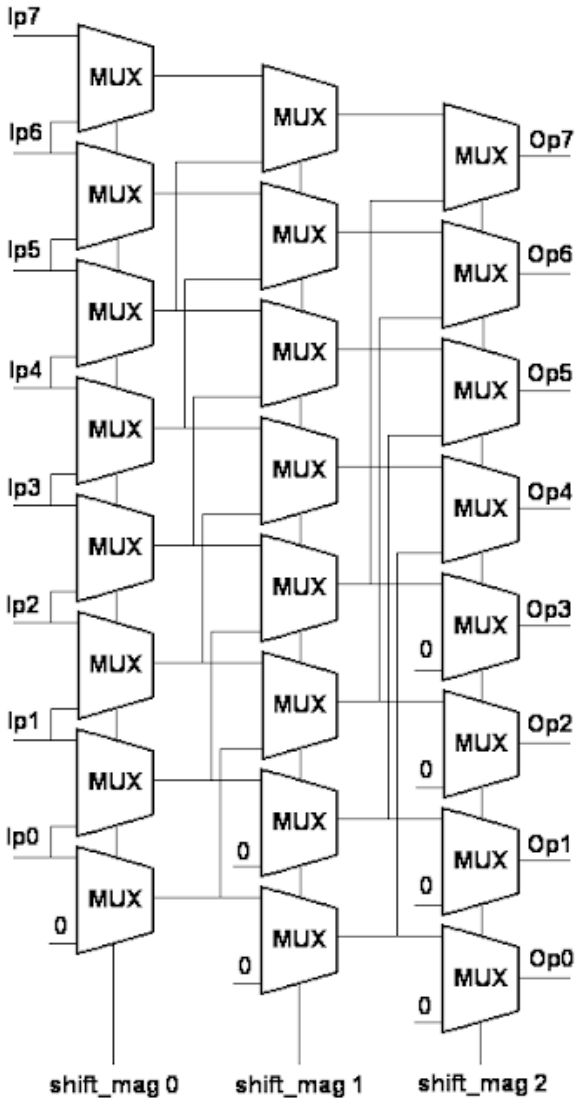

Fig. 4: A smaller 3x8 version of a barrel shifter. However, this does only left shifts.

- **B-type:** 13-bit signed branch offsets (instructions: BEQ, BNE, etc.)
- **U-type:** 20-bit immediates shifted left by 12 bits (instructions: LUI)
- **J-type:** 21-bit signed jump offsets (instruction: JAL)

F. Control Units

Two control modules coordinate instruction execution:

1) Main Control Unit (MCU): Decodes the 7-bit opcode field of the instruction and generates high-level control signals such as:

- ALU operation selection
- Register file write enable
- Data path multiplexing controls
- Memory access controls (for future phases)

2) ALU Control Unit: Takes the 3-bit funct3 field and 7-bit funct7 field from the instruction and generates 4-bit ALU function codes. This implements the secondary decoding layer required for instructions with multiple variants.

III. Implementation Details

A. Module Interfaces

All modules are designed with clear input/output interfaces to enable modular testing and integration:

- Modules use consistent 64-bit data widths for operands and results
- Control signals are clearly named to indicate their function
- Clock and reset signals are provided to all sequential components
- Each module can be independently tested with dedicated testbenches

B. Instruction Set Support

The Phase 1 implementation supports a subset of RV64I base instructions, including:

- **Arithmetic:** ADD, ADDI, SUB, SUBI
- **Logic:** AND, ANDI, OR, ORI, XOR, XORI
- **Shift:** SLL, SLLI, SRL, SRLI, SRA, SRAI
- **Load/Store:** LW, SW (basic support)

IV. Testing and Verification

Each module includes comprehensive testbenches written in Verilog to verify correct operation:

A. Individual Module Testing

Testbenches have been developed for each functional module:

- **ALU Testbench:** Verifies all arithmetic operations, logic operations, and shift operations with various input patterns
- **Adder64 Testbench:** Tests 64-bit addition and subtraction with edge cases including overflow scenarios
- **Gate64 Testbench:** Validates bitwise logic operations
- **Shift64 Testbench:** Confirms correct shifting behavior for all shift types
- **Register File Testbench:** Tests read and write operations across all registers
- **Immediate Generator Testbench:** Validates immediate extraction and sign extension for all instruction formats
- **Control Unit Testbench:** Verifies opcode decoding and control signal generation
- **PC Testbench:** Tests program counter increment and branch handling

B. Test Coverage

The testbenches include:

- Normal operation cases with representative inputs
- Edge cases such as zero operands, maximum values, and boundary conditions
- Negative operands to verify sign handling
- Cross-module integration tests to ensure correct communication

C. Simulation and Results

Simulations are performed using Icarus Verilog, with waveform outputs captured in .vcd format for detailed analysis of signal behavior. The simulation results confirm correct operation of all modules within their specified functional requirements.

V. Challenges and Solutions

A. 64-bit Arithmetic

Implementing correct carry propagation in the 64-bit adder required careful attention to the ripple-carry logic to ensure timing closure.

B. Instruction Format Decoding

The varying formats of RISC-V instructions necessitated a flexible immediate generator that correctly extracts and sign-extends immediates based on instruction type.

C. Control Signal Coordination

Synchronizing control signals across multiple modules required careful design of the control unit hierarchy to avoid hazards.

VI. Future Work (Phase 2)

While Phase 1 establishes the foundational architecture, Phase 2 will extend the design with:

- Data memory module for load/store operations
- Pipeline stages (fetch, decode, execute, memory, write-back)
- Hazard detection and forwarding logic
- Branch prediction mechanisms
- Additional instruction set extensions (multiply, divide, etc.)

VII. Conclusion

This project successfully demonstrates the design and implementation of a 64-bit RISC-V processor core. The modular architecture allows for clear separation of concerns and facilitates future extensions. All modules have been thoroughly tested and verified to operate correctly. The Phase 1 design provides a solid foundation for implementing a pipelined processor in subsequent phases.

References

[1] S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.
[2] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
[3] DownToTheWires, "Risc-v intro and r-type alu instructions," 2025, accessed: 2025. [Online]. Available: https://www.youtube.com/@DowntotheWires